

THS-II

Energy Management System Reinforcement Learning Pipeline

Toyota THS-II · PPO Agent · Custom Gymnasium · ONNX / Raspberry Pi HIL

Parameter	Value
Vehicle	Toyota THS-II — Power-Split Hybrid (Prius ZVW30)
RL Algorithm	PPO — Proximal Policy Optimisation (Stable-Baselines3)
Physics Engine	modeling1.py v4 — 2ZR-FXE · PSD · MG1/MG2 · NiMH
Gymnasium	Custom env wrapping THSIIController.step()
Drive Cycles	WLTC Class 3 · FTP-75/UDDS · US06
Timeline	3-Day Sprint (Core) + Day 4 CARLA Bonus + Day 5 CAN HIL
HIL Platform	Raspberry Pi 4 — 4 GB RAM [Optional Day 5]
Dashboard	Streamlit web app — live SOC & fuel telemetry

CONFIDENTIAL — All instructions are implementation-ready starting points.

Table of Contents

1. Introduction & Problem Statement
2. State of the Art
3. System Architecture & Solution
4. System Components & Technologies
5. Reinforcement Learning Design
6. DAY 1 · Environment Setup & Physics Verification
7. DAY 2 · Gymnasium Env + PPO Training
8. DAY 3 · Evaluation, ONNX Export & Dashboard
9. DAY 4 [OPTIONAL] · CARLA Live Co-Simulation Bonus
10. DAY 5 [OPTIONAL] · CAN Bus Hardware-in-the-Loop
11. Project Deliverables
12. Conclusion & Future Work

1 · Introduction & Problem Statement

The Toyota Hybrid System II (THS-II), deployed across the Prius and Lexus families, uses a planetary power-split device that allows the internal combustion engine (ICE), Motor-Generator 1 (MG1), and Motor-Generator 2 (MG2) to operate simultaneously or independently. This architecture makes energy management a complex, multi-dimensional optimisation problem that is poorly served by static rule tables.

This project replaces conventional hand-crafted EMS policies with a Reinforcement Learning agent trained entirely on the high-fidelity modeling1.py physics engine — a production-grade THS-II v4 simulator with real BSFC maps, planetary gear dynamics, MG1/MG2 efficiency lookup tables, a NiMH thermal model, DC-DC converter model, and blended braking logic.

1.1 Project Objectives

ID	Objective
OBJ-01	Wrap modeling1.py physics inside a standards-compliant Gymnasium environment
OBJ-02	Train a PPO agent to select optimal driving modes per road segment type
OBJ-03	Minimise total fuel consumption over standardised drive cycles (WLTC, FTP-75, US06)
OBJ-04	Maintain battery SOC within $\pm 5\%$ of the 60 % charge-sustaining reference
OBJ-05	Export the trained policy to ONNX; validate inference on Raspberry Pi 4
OBJ-06	Expose results through an interactive Streamlit web dashboard
OBJ-07	[Optional D4] Integrate RL mode output with live CARLA traffic co-simulation
OBJ-08	[Optional D5] Validate ONNX policy in CAN Bus Hardware-in-the-Loop on RPi 4

1.2 Scope & Constraints

Item	Detail
Physics Engine	modeling1.py v4 — 2ZR-FXE, PSD, MG1/MG2, NiMH — DO NOT MODIFY
Drive Modes	EV · ECO · NORMAL · PWR (4 discrete actions, DriveMode enum)
Drive Cycles	WLTC Class 3 · FTP-75/UDDS · US06 (.csv files)
Timeline	3 days core + 1 optional CARLA day + 1 optional CAN HIL day
HIL Platform	Raspberry Pi 4 (4 GB RAM) + MCP2515 CAN HAT [Optional D5]
Budget	~€60 hardware (RPi 4 + CAN HAT)

2 · State of the Art

Rule-Based EMS

Early EMS implementations use deterministic switching rules derived from expert knowledge (charge-depleting / charge-sustaining strategies, thermostat control). They are computationally cheap but cannot account for route topology or traffic context and require manual calibration per vehicle variant.

Optimal Control Methods

Dynamic Programming (DP) yields globally optimal policies over a known drive cycle but requires full future trip knowledge — making it acausal. Pontryagin's Minimum Principle and ECMS reduce cost but rely on heuristic co-state estimation. MPC adds a rolling prediction horizon at the cost of solving a quadratic programme every timestep.

Learning-Based EMS — This Work

PPO-based agents have shown 12–15 % fuel reductions on standard cycles with SOC deviations below 3 %. The distinguishing contribution here is the use of the high-fidelity modeling1.py engine with real lookup tables — not a toy model — and a low-cost HIL pipeline that exports to ONNX for real-time inference on a Raspberry Pi 4, replicable without proprietary hardware.

3 · Proposed Architecture & Solution

The system is structured around two mandatory phases (SIL training + evaluation) and two optional extension phases (CARLA co-simulation on Day 4, CAN HIL on Day 5).

3.1 Phase Overview

Phase	Description
Phase 1 — Training (PC)	Gymnasium env wrapping modeling1.py → PPO agent learns mode-switching → SB3 checkpoint saved
Phase 2 — Evaluation (PC)	SIL validation on all 3 drive cycles → ONNX export → Streamlit dashboard
Phase 3 — CARLA [Opt D4]	RL mode output replaces rule selector inside CarlaTHSISimulation subclass
Phase 4 — CAN HIL [Opt D5]	ONNX policy runs on RPi 4; states/modes exchanged via CAN Bus 2.0B at 500 kbit/s

3.2 Driving Modes — Aligned with modeling1.py DriveMode Enum

Action	Behaviour	Target Segment
0 — ECO	Conservative regen, reduced EV threshold, low ICE load	Urban stop-go
1 — NORMAL	Balanced ICE/motor torque split, standard regen	Mixed roads
2 — PWR	Higher ICE output, aggressive MG2 boost, relaxed SOC target	Highway / grade
3 — EV	Pure electric below 72 km/h, SOC ≥ 45 % required	Low-speed urban

3.3 Training vs. HIL Flow Comparison

Aspect	Training Phase	HIL Validation [Optional]
Environment	Gymnasium wrapping modeling1.py	Same Gym sim on PC + RPi CAN
Agent location	PC — SB3 training loop	PC sends states via CAN frame
Inference	PC — policy.predict()	Raspberry Pi 4 — ONNX Runtime
Timestep	Variable (fast rollout)	Fixed 100 ms CAN frame cycle
Purpose	Learn optimal policy	Validate embedded deployment

4 · System Components & Technologies

4.1 The modeling1.py Physics Engine

modeling1.py (THS-II v4) is the powertrain physics core. Its THSIIController class exposes a .step(throttle, brake, speed, grade, dt) method returning a full telemetry dictionary every timestep. The Gymnasium environment wraps this controller directly — no simplified approximations.

Module / Constant	Value / Description
ICE Model	2ZR-FXE Atkinson 73 kW — BSFC map (RPM × Torque), friction FMEP, coolant thermal
Power Split Device	Planetary K=2.6, dual-mesh efficiency 0.980
MG1 (Sun gear)	42 kW peak, 10 000 rpm limit, 7×7 efficiency table
MG2 (Ring/Output)	60 kW peak, 207 Nm, reduction 2.636×3.267, 7×7 efficiency table
HV Battery	NiMH 201.6 V / 6.5 Ah, RC thermal model, OCV-SOC curve, SOC 40–80 %
DC-DC Converter	500 V HV bus, boost η =0.972, buck η =0.968
Blended Braking	Hydraulic fade-in above 0.3 g decel, regen priority
EV Mode Cap	72 km/h (20 m/s), SOC ≥ 45 % required
ICE Anti-Hunt	Dual threshold P_on=4 kW / P_off=1.5 kW + 10 s minimum warmup hold

4.2 Custom Gymnasium Environment

The environment wraps THSIIController. The state space exposes 5 normalised variables; the action space maps to the 4 DriveMode values. The reward penalises fuel consumption and SOC deviation while rewarding regenerative energy capture.

State Vector (5 dimensions):

Variable	Unit	Source	Description
velocity	m/s	speed arg / drive cycle	Current vehicle speed (normalised ÷ 30)
SOC	[0,1]	state.soc_pct / 100	Battery state-of-charge
road_grade	rad	grade arg	Segment slope (normalised ÷ 0.3)
segment_type	{0,1,2}	derived from speed	0=urban, 1=rural, 2=highway (÷ 2)
acceleration	m/s²	$\Delta\text{speed}/dt$	Instantaneous acceleration (÷ 5)

Reward Function:

Term	Expression	Purpose
Fuel penalty	- out["fuel_rate_gs"]	Minimise instantaneous fuel rate
SOC penalty	- 10.0 × (soc - 0.60)²	Keep SOC at 60 % reference
Regen bonus	+ 0.5 × max(0, -out["p_batt_kw"])	Reward regenerative braking capture
Episode termination	soc < 0.40 OR drive cycle exhausted	Hard SOC floor — matches modeling1.py

4.3 PPO Hyperparameters

Parameter	Value
Algorithm	PPO — Proximal Policy Optimisation
Policy Network	MlpPolicy — 2×64 hidden layers, tanh activation
Learning Rate	3×10 ⁻⁴ with linear decay
n_steps	2048

Batch Size	64
n_epochs	10
Clip Range	0.2
GAE Lambda	0.95
Discount γ	0.99
Total Timesteps	500 000
Train Time	~10 min on laptop CPU (~6 min with GPU)

4.4 Technology Stack

Component	Role
Python 3.11	Core language
modeling1.py (v4)	High-fidelity THS-II powertrain physics engine
Gymnasium 0.29	RL environment API wrapper
Stable-Baselines3 2.x	PPO training framework
PyTorch 2.x	Neural network backend
ONNX Runtime 1.17	Embedded inference on RPi 4 [Optional Day 5]
python-can 4.x	CAN Bus communication [Optional Day 5]
NumPy / Pandas	Numerical computation & drive cycle data I/O
Matplotlib / Plotly	Result visualisation & publication figures
Streamlit 1.x	Interactive web dashboard
CARLA 0.9.x	Live traffic co-simulation [Optional Day 4]

5 · Reinforcement Learning Design

5.1 MDP Formulation

Element	Definition
State S	5-dimensional vector: [velocity, SOC, road_grade, segment_type, acceleration]
Action A	Discrete {0:ECO, 1:NORMAL, 2:PWR, 3:EV} — mirrors DriveMode enum exactly
Reward R	$-\text{fuel_rate_gs} - \lambda \cdot (\text{SOC} - 0.60)^2 + \gamma \cdot \text{regen_energy}$ (per timestep)
Transition T	THSIIController.step() — full powertrain dynamics, no approximation
Episode	One full drive cycle (WLTC: 1800 s · FTP-75: ~2474 s · US06: 596 s)
Discount γ	0.99 — long-horizon fuel and SOC shaping

5.2 Drive Cycle Segmentation Strategy

Each drive cycle is pre-segmented using speed thresholds that mirror the standalone driver logic already present in modeling1.py, enabling predictive mode switching without lookahead:

Speed Range	Segment	Expected Mode
$v < 15$ km/h	Urban (segment_type = 0)	ECO or EV preferred
$15 \leq v < 80$ km/h	Rural (segment_type = 1)	NORMAL preferred
$v \geq 80$ km/h	Highway (segment_type = 2)	PWR or NORMAL preferred

5.3 Reward Tuning Guidelines

Symptom	Likely Cause	Fix
SOC drops below 45 %	lambda_soc too low	Increase lambda_soc: 10 → 15 → 20
Agent always selects EV mode	Regen bonus too high	Reduce gamma_regen: 0.5 → 0.2
Reward flat after 100k steps	Learning rate too high	Decay LR: 3e-4 → 1e-4
Fuel not improving vs baseline	Action not applied to ems	Verify DriveMode is set in step()
Episode terminates too early	SOC cutoff too strict	Check the soc < 0.40 condition
Mode oscillation every few steps	Insufficient hysteresis	Add mode hold counter (≥ 3 steps)

5.4 Evaluation Metrics

Metric	Definition	Target
Fuel Savings (%)	RL vs rule-based baseline on test cycle	> 5 %
SOC Deviation	RMSE of SOC from 60 % across full episode	< ± 5 %
Mode Distribution	Histogram of mode selections per segment type	Qualitative
Inference Latency	ONNX sess.run() time on RPi 4 [Optional HIL]	< 10 ms
CAN Reliability	Dropped / corrupt frames [Optional HIL]	< 0.1 %
RAM Footprint	Peak RAM on RPi during HIL [Optional HIL]	< 200 MB
CPU Usage	Average CPU load on RPi [Optional HIL]	< 30 %

DAY 1 · MON — Environment Setup & Physics Verification

Foundation · All subsequent days depend on this

Day 1 is non-negotiable. Every downstream task assumes a verified, fully-functional modeling1.py standalone run and a properly configured Python environment. Do not proceed to Day 2 until all checkpoints below are green.

Part 1A · Python Stack Installation

- Create a dedicated virtual environment (Python 3.11 recommended) and activate it before installing anything.
- Install the RL and simulation stack in a single pass: stable-baselines3[extra], gymnasium==0.29, torch (CPU build unless you have CUDA), onnxruntime, tensorboard.
- Install the data and visualisation stack: numpy, pandas, matplotlib, plotly, streamlit.
- Install optional packages now so they do not break your environment later: python-can (for CAN HIL), carla (only if CARLA is installed on your machine).
- Freeze requirements into requirements.txt immediately after installation so the environment is reproducible.
- Verify the installation by importing each major package in a Python REPL — any ImportError must be resolved before continuing.

Part 1B · Project Directory Structure

Set up the following directory layout at the project root. All subsequent instructions reference these paths — deviation will break relative imports:

Path	Purpose
env/	Gymnasium environment module (ths_env.py + drive cycle CSVs)
env/drive_cycles/	WLTC.csv, FTP75.csv, US06.csv — speed_ms column required
training/	PPO training script, baseline comparator, export script
eval/	SIL evaluation, plotting, and metrics scripts
models/	Saved SB3 checkpoints and ONNX model output
hil/	CAN Bus PC-side and RPi controller scripts [Optional Day 5]
carla/	CARLA RL bridge subclass [Optional Day 4]
app/	Streamlit dashboard script
tb_logs/	TensorBoard log directory (auto-created by training script)
modeling1.py	THS-II physics engine — place at project root, DO NOT MODIFY

Part 1C · Physics Verification — modeling1.py Standalone Run

Before writing a single line of Gymnasium code, verify that modeling1.py runs correctly in standalone mode. This confirms the physics engine, BSFC tables, and KPI output are intact.

- Run modeling1.py with the --standalone flag. Confirm it prints KPI values and exits without errors.
- Run again with --standalone --ftp75. All 2474 timesteps must complete. Note the final total_fuel_g and soc_final values for later comparison.
- Run with --standalone --plot. Verify that the 4×2 subplot PNG (BSFC operating point, SOC trace, RPM, power split) is saved to /tmp/ and all subplots are populated.
- Open ths2_kpis.csv (output in project root). Confirm it has 28 columns and zero NaN values in the SOC or fuel_rate_gs columns.
- Inspect the THSIIController class signature: note the init_drive_mode parameter, the state dataclass fields, and the exact keys returned by .step() — you will use these in Day 2.

- Note the DriveMode enum values (ECO, NORMAL, PWR, EV, AUTOMATIC) and confirm they match the 4-action mapping planned for the RL environment.
- Document the standalone fuel total for WLTC and FTP-75 — this becomes your physics-engine baseline for comparison on Day 3.

Part 1D · Drive Cycle Data Preparation

- Download WLTC Class 3 from the official UNECE source. Convert to CSV with a single column: speed_ms (vehicle speed in m/s, 1-second resolution, 1800 rows).
- Download FTP-75/UDDS from the US EPA. Same format: speed_ms column, ~2474 rows.
- Download US06 from the US EPA. Same format: speed_ms column, 596 rows.
- Place all three files in env/drive_cycles/ and verify row counts match expected values before continuing.
- Optionally add a road_grade_rad column (zeros for flat cycle) to each CSV now — the environment can read it later.

Day 1 — Checkpoints (All Must Pass)

- ✓ python modeling1.py --standalone → prints KPIs, zero errors
- ✓ python modeling1.py --standalone --ftp75 → 2474 steps complete, fuel_g and soc_final printed
- ✓ --plot flag → 4×2 subplot PNG saved, all 8 subplots populated
- ✓ ths2_kpis.csv → 28 columns, no NaN in SOC or fuel_rate_gs
- ✓ WLTC.csv / FTP75.csv / US06.csv present in env/drive_cycles/ with correct row counts
- ✓ All Python imports (SB3, Gymnasium, PyTorch, ONNX, Streamlit) succeed in REPL

Day 2 builds the two most critical software artefacts: the custom Gymnasium environment that wraps modeling1.py, and the PPO training script. Both must be verified and producing improving reward curves before end of day.

Part 2A · Implementing env/th_s_env.py

Create the THSEnv class in env/th_s_env.py by implementing each Gymnasium method in order:

- `__init__`: Accept cycle name ('WLTC', 'FTP75', 'US06') and `dt=0.1` parameters. Define `observation_space` as a `Box(5,)` with `low=0`, `high=1`, `dtype=float32`. Define `action_space` as `Discrete(4)`. Store cycle path for use in `reset()`.
- `reset()`: Instantiate a new `THSIIController`(`init_drive_mode=DriveMode.ECO`) each call — this ensures the physics engine is fully re-initialised each episode. Load the drive cycle CSV, extract the `speed_ms` column as a NumPy array, reset the step counter `idx=0` and `self.speed=0.0`, then return the initial observation and an empty info dict.
- `step()`: Receive the integer action (0–3). Map it to the corresponding `DriveMode` enum value. Set `ems.state.selector_mode` to the chosen mode. Compute throttle and brake from the difference between current speed and target cycle speed. Call `ems.step(throttle, brake, speed, grade, dt)`. Extract `soc`, `fuel_rate_gs`, and `p_batt_kw` from the returned telemetry dict. Compute reward. Advance `idx`. Return (`obs`, `reward`, `done`, `truncated=False`, `info=out`).
- `_obs()`: Build and return the 5-element normalised observation array: [`speed/30`, `soc_pct/100`, `grade/0.3`, `segment_type/2`, `accel/5`]. Compute `segment_type` from current speed using the thresholds defined in the MDP section.
- Termination logic: set `done=True` when `idx` reaches the end of the drive cycle array OR when `soc` drops below 0.40 (matches modeling1.py SOC floor).
- Ensure the environment does NOT import CARLA or pygame at module level — use lazy imports or guard with `try/except` so the environment works without those packages.

Part 2B · Environment Validation with check_env

- Run Gymnasium's built-in `check_env(THSEnv('WLTC'))` before writing any training code. Resolve every warning — common issues are observation dtype mismatch, reward not a float scalar, or `step()` returning wrong number of values.
- Run one complete manual episode with random actions. Plot `SOC` and `fuel_rate_gs` vs `timestep`. Verify `SOC` stays within `[0.40, 0.80]` and reward is always negative with magnitude below 2 g/s.
- Confirm the `done` flag is raised cleanly at step 1800 for WLTC without an `IndexError` on the cycle array.
- Print the `DriveMode` being applied each step and confirm it changes when different actions are passed — this catches the common bug where mode is set on the wrong attribute.

Part 2C · Rule-Based Baseline (training/baseline_rule.py)

Implement a deterministic baseline before training so you have a reference fuel total for comparison:

- Instantiate `THSEnv('WLTC')` and run one complete episode with the rule: ECO if $v < 15$ km/h, NORMAL if $v < 80$ km/h, PWR otherwise.
- Record total fuel consumed (sum of `fuel_rate_gs` × `dt` over all steps) in grams.
- Record the SOC trajectory. Print mean SOC deviation from 60 %.
- Save these numbers — your Day 3 PPO agent must beat this fuel total by at least 5 %.

Part 2D · PPO Training (training/train_ppo.py)

- Instantiate `THSEnv('WLTC')` as the training environment and a separate `THSEnv('WLTC')` as the evaluation environment — never share instances between train and eval.
- Configure PPO with the hyperparameters from Section 4.3: `learning_rate=3e-4`, `n_steps=2048`, `batch_size=64`, `n_epochs=10`, `gamma=0.99`, `gae_lambda=0.95`, `clip_range=0.2`, `tensorboard_log='./tb_logs/'`.
- Add an `EvalCallback` with `eval_freq=50_000` steps and `best_model_save_path='./models/'`. This saves the best checkpoint automatically — not just the final one.

- Call `model.learn(total_timesteps=500_000)`. Monitor the TensorBoard `ep_rew_mean` curve in a separate terminal; it should begin rising within the first 100k steps.
- If the reward curve is flat after 150k steps, apply the reward tuning guidelines from Section 5.3 before continuing to more timesteps.
- After training, save the final model as `models/th_s_agent_final`. The best checkpoint from `EvalCallback` is saved separately as `models/best_model.zip` — use this for evaluation.

Day 2 — Checkpoints (All Must Pass)

- ✓ `check_env(THSEnv("WLTC"))` → zero warnings or errors
- ✓ Random episode completes 1800 steps, done flag raised cleanly, no crash
- ✓ SOC stays in `[0.40, 0.80]` across random episode — plot confirms
- ✓ Fuel reward is always negative with magnitude `< 2 g/s`
- ✓ Baseline rule total fuel recorded in grams (reference for Day 3 comparison)
- ✓ PPO training starts and TensorBoard `ep_rew_mean` is rising or at least not flat by step 100k
- ✓ `models/best_model.zip` exists after `EvalCallback` fires at 50k steps

Day 3 completes the mandatory pipeline. By end of day you will have verified fuel savings, a deployable ONNX model, and a Streamlit dashboard. All subsequent days (CARLA, CAN HIL) are optional extensions — the project is fully deliverable after Day 3.

Part 3A · Software-in-the-Loop Evaluation (`eval/sil_evaluate.py`)

- Load `models/best_model.zip` and run deterministic evaluation (`deterministic=True`) on all three drive cycles: WLTC, FTP-75, US06. Run each at least 3 times and average results to reduce stochastic variance.
- For each cycle, record: total fuel consumed (g), final SOC, SOC RMSE from 60 %, episode return, and mode selection counts per segment type.
- Compare RL total fuel against the rule-based baseline total (from Day 2) and against a random-action baseline. Compute percentage fuel savings for each cycle.
- Verify the primary gate: RL fuel savings on WLTC must exceed 5 % vs the rule-based baseline. If not, revisit `lambda_soc` and `gamma_regen` before proceeding.
- Generate publication-quality plots (300 dpi): SOC trajectory over time, cumulative fuel vs time, mode selection histogram stacked by segment type, reward per episode during training.
- Generate a fuel comparison bar chart: three groups (WLTC, FTP-75, US06), three bars each (Random, Rule-based, RL PPO). Save all figures to `eval/figures/`.

Part 3B · ONNX Export (`training/export_onnx.py`)

- Load `models/best_model.zip` using `PPO.load()`. Access the underlying policy network.
- Create a dummy input tensor of shape (1, 5) with dtype `float32` — this matches the 5-dimensional observation space.
- Export using `torch.onnx.export` with `opset_version=17`, `input_names=['obs']`, `output_names=['action_logits']`. Save to `models/th_s_policy.onnx`.
- Verify the ONNX file exists and is approximately 200 KB. Files significantly larger indicate the wrong model object was exported.
- Validate the export by running ONNX Runtime inference on the same dummy input and confirming the argmax of output logits matches the SB3 model's prediction for the same input.
- Benchmark ONNX inference on your PC: run 1000 sequential `sess.run()` calls and compute mean latency. Target < 2 ms per call on PC.
- If you have a Raspberry Pi available: copy `th_s_policy.onnx` via scp or USB. Run the same 1000-call benchmark on the RPi. Target < 5 ms per call. Record actual latency for the Day 5 HIL report.

Part 3C · Streamlit Dashboard (`app/streamlit_dashboard.py`)

- Build the main dashboard layout with a sidebar for selecting drive cycle (WLTC / FTP-75 / US06) and agent mode (RL PPO / Rule-based / Random).
- Implement a 'Run Episode' button that executes one deterministic episode and stores the per-step telemetry in `st.session_state`.
- Display a real-time SOC trajectory chart (line chart, SOC % vs timestep) with a horizontal reference line at 60 %.
- Display a cumulative fuel consumption chart (g vs timestep) with three lines: RL, rule-based, and random baselines.
- Display a mode selection donut chart showing the percentage of timesteps spent in each `DriveMode`.
- Display a summary metrics table: total fuel (g), fuel savings (%), SOC RMSE, episode duration (s).
- Add a live replay mode that re-reads `th_s2_kpis.csv` row by row and updates charts at 10 Hz using `st.empty()` — this simulates real-time telemetry.
- Test that the dashboard launches without errors via `'streamlit run app/streamlit_dashboard.py'` and all charts populate after pressing Run Episode.

Part 3D · Git Release & Documentation

- Write a comprehensive README.md covering: project overview, installation steps, Day 1-3 run instructions, description of all output files, and how to interpret the dashboard.
- Add an architecture diagram to the README (ASCII or image) showing the data flow from drive cycle CSV → THSIIController → Gymnasium → PPO → ONNX.
- Commit all code, models/, eval/figures/, and requirements.txt. Tag the release as v1.0-sil.
- Archive the models/ and eval/ directories to a ZIP for submission or sharing.

Day 3 — Checkpoints (Core Project Complete)

- ✓ RL fuel savings on WLTC > 5 % vs rule-based baseline — confirmed in SIL evaluation
- ✓ SOC RMSE < ± 5 % of 60 % across all three drive cycles
- ✓ models/thi_policy.onnx exists, ~200 KB, ONNX Runtime inference validated
- ✓ ONNX inference latency < 2 ms per call on PC
- ✓ eval/figures/ contains SOC trace, fuel bar chart, mode histogram — all at 300 dpi
- ✓ Streamlit dashboard launches and all charts populate correctly
- ✓ Git tag v1.0-sil committed with all deliverables

DAY 4 · THU [OPTIONAL] — CARLA Live Co-Simulation Bonus

Optional bonus — requires CARLA 0.9.x installed on host

■ **This entire day is optional.** Days 1–3 constitute a complete, deliverable project. Proceed to Day 4 only if you have CARLA 0.9.x installed and a working CARLA server connection. The RL policy has no dependency on CARLA and does not need to be retrained for this integration.

Part 4A · Understanding CarlaTHSISimulation in modeling1.py

- Read the CarlaTHSISimulation class in modeling1.py carefully. Identify the `_select_drive_mode()` method — this is the only method you will override. Do not modify any other physics logic.
- Note how the base class calls `_select_drive_mode()` on each tick: it receives (speed, soc, grade, seg) as arguments and must return a DriveMode enum value.
- Identify which CARLA actor and sensor handles are created by the base class so you do not duplicate them in your subclass.
- Confirm that importing modeling1.py with CARLA unavailable raises an ImportError — your bridge script must guard against this.

Part 4B · Building the RL Bridge (carla/carla_rl_bridge.py)

- Create a subclass RLBridgeSimulation that inherits from CarlaTHSISimulation.
- In `__init__`, call `super().__init__()` and then load the trained PPO policy from `models/best_model.zip` using `PPO.load()`. Store it as `self.policy`.
- Override `_select_drive_mode(speed, soc, grade, seg)`: construct the 5-element observation array using the same normalisation as THSEnv (speed/30, soc/100, grade/0.3, seg/2, 0.0 for acceleration). Call `self.policy.predict(obs, deterministic=True)` and map the integer action to the corresponding DriveMode enum value. Return the DriveMode.
- Add a logging hook: each time `_select_drive_mode` is called, append the timestep, chosen DriveMode, current SOC, and speed to a list. After the simulation ends, save this list to `carla/carla_rl_log.csv` for comparison with standalone SIL results.
- In the `__main__` block, instantiate RLBridgeSimulation with CARLA host/port arguments and call `sim.run()`. Add a `--map` argument defaulting to Town03.

Part 4C · Running and Validating the CARLA Integration

- Start the CARLA server (`./CarlaUE4.sh`) and verify it is accepting connections on port 2000 before running the bridge script.
- Run `python carla/carla_rl_bridge.py --map Town03`. Confirm the CARLA vehicle spawns, moves, and that mode decisions appear in the terminal log.
- Monitor that SOC stays within the expected range during the CARLA run — if the CARLA physics differ significantly from the standalone cycle, mode selection may be suboptimal but should remain stable.
- After the run completes, load `carla_rl_log.csv` and compare the mode distribution histogram against the SIL results from Day 3. Document any significant differences.
- Compare fuel consumption estimated from the CARLA run against the standalone WLTC result. Note: CARLA traffic and physics variability mean exact comparison is not expected — qualitative consistency is the goal.
- Optionally record a short video of the CARLA HUD with the mode indicator visible — include it in the final project demo.

Day 4 — Optional Checkpoints

- ✓ [Optional] CARLA server connects without error and vehicle spawns in Town03
- ✓ [Optional] RLBridgeSimulation._select_drive_mode() called each tick — mode printed to terminal
- ✓ [Optional] carla/carla_rl_log.csv populated with timestep, mode, SOC, speed columns
- ✓ [Optional] Mode distribution in CARLA qualitatively consistent with SIL results
- ✓ [Optional] No Python exceptions during a full 5-minute CARLA run

DAY 5 · FRI [OPTIONAL] — CAN Bus Hardware-in-the-Loop

Optional HIL — requires RPi 4 + MCP2515 CAN HAT

■ **This entire day is optional.** The SIL pipeline (Days 1–3) is fully complete and sufficient. Proceed to Day 5 only if you have a Raspberry Pi 4 (4 GB), an MCP2515 CAN HAT, and a USB-CAN adapter for the PC. The ONNX policy from Day 3 is used as-is — no retraining required.

Part 5A · Hardware Assembly & OS Configuration

- Mount the MCP2515 CAN HAT on the Raspberry Pi 4. Connect the PC USB-CAN adapter (Canable or PCAN-USB) to a twisted-pair cable wired to the HAT. Place 120 Ω termination resistors at both bus ends.
- Flash Raspberry Pi OS (64-bit, Bookworm) to the RPi SD card if not already done. Enable SPI in raspi-config (Interface Options → SPI).
- Add the MCP2515 overlay to /boot/firmware/config.txt: dtoverlay=mcp2515-can0,oscillator=8000000,interrupt=25. Reboot and verify can0 appears in 'ip link show'.
- Bring up the CAN interface on both PC and RPi: `sudo ip link set can0 type can bitrate 500000 && sudo ip link set can0 up`. Confirm the interface is UP with 'ip -s link can0'.
- Perform a loopback test: send a test frame from the PC using cansend and receive it on the RPi using candump. Confirm correct frame content before proceeding.

Part 5B · CAN Protocol — Frame Definitions

Frame 0x100 — Vehicle State (PC → RPi) · DLC: 8 bytes · Cycle: 100 ms

Bytes	Signal	Type	Encoding	Range	Resolution
0–1	velocity	uint16	$v \times 100$ [cm/s]	0–8000	0.01 m/s
2	SOC	uint8	$SOC \times 100$ [%]	0–100	1 %
3	road_grade	int8	$grade \times 50$ [°]	-90–90	0.02°
4	segment_type	uint8	0/1/2	0–2	—
5	acceleration	int8	$a \times 10$ [dm/s ²]	-50–50	0.1 m/s ²
6	frame_counter	uint8	Rolling 0–255	0–255	—
7	checksum	uint8	XOR bytes 0–6	0–255	—

Frame 0x200 — Mode Decision (RPi → PC) · DLC: 4 bytes · Triggered on 0x100 receipt

Bytes	Signal	Type	Encoding	Range
0	mode_action	uint8	0=ECO / 1=NORM / 2=PWR / 3=EV	0–3
1	latency_ms	uint8	Inference time in ms	0–255
2	frame_counter	uint8	Echo of received frame counter	0–255
3	checksum	uint8	XOR bytes 0–2	0–255

Part 5C · PC-Side CAN Driver (hil/pc_side_can.py)

- Implement `encode_state_frame()`: pack the 5 observation values into the 8-byte 0x100 frame using struct.pack. Apply XOR checksum over bytes 0–6.
- Implement `send_state()`: take the current THSEnv observation array and frame counter, encode it, and transmit via `bus.send()` on can0 at CAN-ID 0x100.
- Implement a `receive_mode()` function that calls `bus.recv(timeout=0.05)` and waits for a 0x200 frame. Decode the action byte and latency. Validate XOR checksum. On timeout, return the previous mode (fail-safe hold).
- Build the main HIL loop: for each timestep in the WLTC cycle, send the state frame, wait for the mode response, apply the received DriveMode to the Gymnasium environment, advance one step, log timestep/mode/latency/reward/SOC to

hil/hil_log.csv.

Part 5D · RPi-Side ONNX Controller (hil/rpi_controller.py)

- On the Raspberry Pi, copy ths_policy.onnx to ~/models/ via scp or USB.
- Install ONNX Runtime on the RPi: `pip install onnxruntime` (CPU build, no CUDA needed on RPi 4).
- Implement the main receive loop: block on `bus.recv()` for 0x100 frames. On receipt, decode the 8-byte state payload using `struct.unpack`. Reconstruct the float32 observation array by reversing the encoding (divide by scaling factors).
- Run ONNX inference: call `sess.run(None, {input_name: obs})` where `obs` is shape (1,5). Take argmax of the output logits to get the action integer (0–3). Map to mode name for logging.
- Measure inference latency using `time.perf_counter()` around the `sess.run()` call. Include the latency in byte 1 of the 0x200 response frame.
- Encode and send the 0x200 mode frame immediately after inference. Log mode and latency to a CSV on the RPi for later analysis.
- Implement a watchdog: if no 0x100 frame is received within 1 second, print a warning and hold the last mode — this prevents runaway behaviour on CAN bus loss.

Part 5E · HIL Validation Run (eval/hil_evaluate.py)

- Start `rpi_controller.py` on the Raspberry Pi first, then start `pc_side_can.py` on the PC.
- Run a full WLTC episode (1800 steps × 100 ms = 180 s real time). Monitor the terminal for any CAN frame errors or checksum failures.
- After the run, load `hil_log.csv` on the PC. Compute mean and max inference latency. Verify mean < 5 ms and max < 10 ms.
- Compare HIL total fuel, SOC trajectory, and mode distribution against the SIL results from Day 3. Document any discrepancies in the project report.
- Compute CAN frame error rate: $(\text{dropped} + \text{corrupt frames}) / \text{total frames} \times 100$. Target < 0.1 %.
- Monitor RPi resource usage during the run using `htop`: peak RAM should be < 200 MB and average CPU < 30 % — confirm with a screenshot or log.

CAN Bus Timing & Error Handling Reference

Parameter	Value	Note
Frame period (0x100)	100 ms	10 Hz control loop — matches RPi inference budget
Max inference latency	< 10 ms	Measured on RPi 4 CPU only
CAN frame timeout	50 ms	<code>bus.recv(timeout=0.05)</code> — fail-safe triggers on miss
Missing frame action	Hold previous	Prevent mode switching on dropped frame
Bus-off recovery	Automatic	MCP2515 built-in hardware recovery
Error frame target	< 0.1 %	Monitor via <code>ip -s link can0</code>

Day 5 — Optional Checkpoints

- ✓ [Optional] CAN loopback test passes — `cansend` on PC received cleanly by `candump` on RPi
- ✓ [Optional] `ths_policy.onnx` ONNX inference benchmark on RPi < 5 ms per call
- ✓ [Optional] Full WLTC HIL episode completes 1800 steps without CAN error
- ✓ [Optional] Mean HIL inference latency < 5 ms, max < 10 ms — confirmed in `hil_log.csv`
- ✓ [Optional] CAN frame error rate < 0.1 % over full episode
- ✓ [Optional] RPi peak RAM < 200 MB, average CPU < 30 % — confirmed via `htop`
- ✓ [Optional] HIL fuel and SOC results consistent with SIL results from Day 3

11 · Project Deliverables

Software Deliverables

ID	Path	Description	Status
SW-01	env/ths_env.py	Custom Gymnasium environment wrapping modeling1.py	Mandatory
SW-02	modeling1.py	THS-II v4 physics engine — provided, do not modify	Mandatory
SW-03	env/drive_cycles/*.csv	WLTC, FTP-75, US06 drive cycle data	Mandatory
SW-04	training/train_ppo.py	PPO training script with EvalCallback	Mandatory
SW-05	training/baseline_rule.py	Rule-based EMS comparator for fuel baseline	Mandatory
SW-06	training/export_onnx.py	PyTorch policy → ONNX export and validation	Mandatory
SW-07	eval/sil_evaluate.py	SIL validation on all 3 cycles + plot generation	Mandatory
SW-08	app/streamlit_dashboard.py	Interactive web dashboard for telemetry replay	Mandatory
SW-09	models/best_model.zip	Best SB3 PPO checkpoint from EvalCallback	Mandatory
SW-10	models/ths_policy.onnx	Optimised ONNX model for embedded deployment	Mandatory
SW-11	carla/carla_rl_bridge.py	CARLA ↔ RL mode bridge via RLBridgeSimulation subclass	Optional D4
SW-12	hil/pc_side_can.py	PC CAN state transmitter / mode receiver	Optional D5
SW-13	hil/rpi_controller.py	RPi ONNX inference + CAN mode responder	Optional D5
SW-14	eval/hil_evaluate.py	HIL loop validation with latency and error logging	Optional D5

Hardware Deliverables (Optional Day 5)

ID	Item	Role
HW-01	Raspberry Pi 4 (4 GB)	HIL target running ONNX inference
HW-02	MCP2515 CAN HAT	SPI-to-CAN interface for Raspberry Pi
HW-03	USB-CAN Adapter	PC-side CAN interface (Canable / PCAN-USB)
HW-04	Twisted-pair cable	Bus wiring with 120 Ω termination at both ends

12 · Conclusion & Future Work

12.1 Summary

This project delivers a complete, reproducible pipeline for learning-based Energy Management in a Toyota THS-II hybrid vehicle. The core contribution — a PPO agent trained in 3 days on standardised drive cycles using the high-fidelity modeling1.py physics engine — learns which driving mode best suits each road segment without any simplified powertrain approximations. The pipeline is validated through Software-in-the-Loop evaluation on WLTC, FTP-75, and US06, with optional CARLA co-simulation on Day 4 and optional CAN Bus HIL on a Raspberry Pi 4 on Day 5.

12.2 Expected Results

Metric	Target	Context
Fuel savings	5–15 %	vs. rule-based baseline on WLTC cycle
SOC deviation	< ±5 %	RMSE from 60 % across all test cycles
ONNX latency (PC)	< 2 ms	Per inference call on laptop CPU
ONNX latency (RPi 4)	< 5 ms	Per inference call on RPi 4 CPU [Optional D5]
CAN frame error rate	< 0.1 %	Over 1800-step WLTC HIL episode [Optional D5]
Dashboard	10 Hz replay	Streamlit app rendering live mode decisions
CARLA integration	Qualitative	RL mode control in live traffic [Optional D4]

12.3 Future Work

ID	Topic	Description	Effort
FW-01	Full CARLA Emergent Traffic	Replace prescribed cycles with dynamic CARLA scenarios	High
FW-02	Multi-Objective Reward	Add comfort (jerk) and emissions (NOx) terms to reward function	Medium
FW-03	Route-Aware Prediction	Google Maps API → segment type prediction before trip start	Medium
FW-04	dSPACE HIL Upgrade	Upgrade HIL platform for automotive-grade certification	High
FW-05	SAC / TD3 Comparison	Benchmark off-policy algorithms for sample efficiency	Low
FW-06	Online Adaptation	Fine-tune policy on-device after deployment in real vehicle	High
FW-07	Fleet Federated Learning	Federated RL across multiple vehicles — shared policy backbone	Research

This document was generated as part of the THS-II EMS RL project. All instructions are implementation-ready starting points — adapt hyperparameters and map data to your specific vehicle calibration.

Bonne chance avec la réalisation. — Oussama Chouaibi · EABA